EE627 Homework 9
PySpark ML Classifiers for Music Recommendation

Jude Eschete

May 4, 2026

Instruction Checklist

- ✓ Download `test2_new.txt` (ground-truth labels for 1,000 users × 6 tracks) . Section 2
- ✓ Use the `userID`s in `test2_new.txt` to fetch related rating scores from the training data Section 2.3
- ✓ Train classifiers using `from pyspark.ml.classification import *` on the labeled rating-score matrix . . . . . Section 3
- ✓ Try four (or more) different PySpark.ML classifiers as shown in lecture . . . . Section 3
- ✓ Apply the trained models to the final-project test data and submit at Kaggle Section 5
- ✓ Kaggle public-score screenshot . . . . . Section 5
- ✓ Attach Python code file to Canvas . . . . . Appendix 8

1 Introduction

This homework closes the loop on the Yahoo Music recommender used throughout the semester: HW6 introduced collaborative filtering, HW7 implemented matrix factorization in MATLAB, HW8 ported MF to PySpark ALS, and now HW9 trains *supervised* classifiers in `pyspark.ml` on the ground-truth labels released as `test2_new.txt`. The shift is conceptually significant. Through HW8 the labels were *implicit*—inferred from rating scores or imputed from heuristics—and every model was either a regressor (MF) or a hand-tuned scoring rule (the v2 heuristic that scored 0.871). With `test2_new.txt` we have 6,000 explicit binary labels for the actual recommendation task, so the same feature pipeline that powered the midterm now feeds a true supervised learning loop.

Five classifiers (instead of the required four) were trained and cross-validated, plus six ensemble blends were generated. The best-performing single classifier and the rank-averaged blend both reached a Kaggle public score of 0.914, surpassing the midterm's best of 0.911.

2 Data Pipeline

2.1 Inputs

- `test2_new.txt` — 6,000 rows in `userID|trackID|label` format, 1,000 users × 6 candidate tracks each, with a balanced 3,000/3,000 split of positives and negatives.
- `trainItem2.txt`, `trackData2.txt`, `albumData2.txt` — 49,204 users with 12,403,575 ratings, 224,041 tracks, and 52,829 albums (same files used in the midterm).
- `testItem2.txt` — the 20,000-user × 6-candidate test set (120,000 rows) used for the Kaggle submission.

2.2 Album-Based Track Enrichment

Roughly 11.7% of tracks have a missing artist field in `trackData2.txt`, and many tracks list zero genres. Each track's album is looked up in `albumData2.txt`; if the track lacks an artist, the album's artist is filled in, and any genres on the album that aren't already on the track are appended. This recovered 8,134 artists and added 119,842 genres, leaving only 18,762 tracks (8.4%) genuinely artist-less. The same enrichment pass was used in the midterm v5 model.

2.3 Feature Engineering

For every (uid, tid) pair in `test2_new.txt`, twenty-five numerical features are computed by inspecting the user's training history. After dropping three features that the first run identified as having 0.000 importance on multiple classifiers (`evidence_count`, `genre_artist_int`, `album_genre_int`), the final feature set was

- *Direct hierarchy ratings* (4): `album_score`, `has_album`, `artist_score`, `has_artist`.
- *Genre statistics* (5): `genre_count`, `genre_max`, `genre_min`, `genre_mean`, `genre_var`.
- *Interaction term* (1): `album_artist_int` $= r_{\text{alb}} \cdot r_{\text{art}}/100$.
- *User profile* (3): `user_mean`, `user_std`, $\log(1 + \texttt{user_n})$.
- *User-relative offsets* (3): scores minus user mean.
- *Global popularity* (3): per-album / artist / genre mean rating.
- *Log-scaled rater counts* (3): controls for the long-tail popularity distribution following the spatial-sign / scaling lesson from Week 12.

- *Collaborative filtering siblings* (2): `cf_score` and $\log(1 + \texttt{cf_count})$ over tracks sharing the same artist or album.
- *v2 heuristic score* (1): the midterm's hand-tuned linear combination $r_{\text{alb}} + r_{\text{art}} + n_{\text{genre}} \cdot \bar{r}_{\text{genre}}/10$, kept as a feature so trees can use it as a strong split candidate.

The same feature function is also applied to all 120,000 rows of `testItem2.txt` so the trained classifiers can score every Kaggle candidate. The features are emitted to `features_labeled.csv` (training) and `features_test.csv` (Kaggle test) to skip recomputation on subsequent runs.

3 Classifiers

3.1 Spark Pipeline

All classifiers consume a single `VectorAssembler`-built feature column. The labeled DataFrame is split 70%/30% for hyperparameter tuning and validation reporting, then the best-performing model is refit on the full 6,000 rows before generating Kaggle predictions. The `BinaryClassificationEvaluator` on `areaUnderROC` drives both validation reporting and the cross-validation grid searches. The Spark session is configured with `-Xss32m` for both driver and executor JVMs because the default 1 MB thread stack overflows when serializing deep `RandomForestClassifier` models on Windows.

3.2 The Five Classifiers

1. **Logistic Regression** with `regParam` and `elasticNetParam` swept by 5-fold CV (9 combinations).

2. **Decision Tree Classifier** with `maxDepth=5` (the lecture-notes default).

3. **Random Forest Classifier** with a 12-combination CV grid: `numTrees` $\in \{200, 500\}$, `maxDepth` $\in \{10, 12, 15\}$, `featureSubsetStrategy` $\in \{\text{sqrt}, \text{log2}\}$.

4. **Gradient-Boosted Tree Classifier (base)** with the lecture-notes defaults (`maxIter`=10, `stepSize`=0.1).

5. **Gradient-Boosted Tree Classifier (CV-tuned)** with a 24-combination CV grid: `maxDepth` $\in \{3, 5\}$, `maxIter` $\in \{50, 100, 200\}$, `stepSize` $\in \{0.05, 0.1\}$, `subsamplingRate` $\in \{0.7, 1.0\}$.

Two more variants were also trained for ensembling experiments: an RF without the `v2_score` feature (to test whether the model can rediscover the v2 signal from raw components), and an Extra-Trees-style RF (`numTrees`=500, `maxDepth`=15, `featureSubsetStrategy`=onethird, `subsamplingRate`=0.8, different seed). All seven models are refit on the full 6,000 rows after CV completes.

3.3 Validation AUC

Table 1: Validation AUC on the held-out 30% of `test2_new.txt`. "CV" denotes cross-validated grid search; "base" uses lecture-notes defaults.

Classifier	Val AUC	Comment
RF without `v2_score`	**0.9472**	24 features (no v2)
Random Forest (Big CV)	0.9468	best: 200 trees, depth 10, sqrt
GBT (Big CV)	0.9454	best: depth 3, 100 iter, 0.1 step
Extra Trees	0.9450	500 trees, depth 15, onethird
GBT (base)	0.9377	defaults, no CV
Logistic Regression CV	0.9376	best: `regParam`=0.01, `l1`=0
Decision Tree	0.7899	single tree, depth 5

The single decision tree is the only model that fails to clear 0.90 on validation AUC; the four tree-ensemble classifiers (RF, RF-no-v2, GBT-CV, ExtraTrees) cluster within 0.0022 AUC of one another. Logistic regression sits roughly 0.01 AUC behind the trees, which is the expected gap when the signal is non-linear in the features.

3.4 Feature Importance

The `v2_score` feature dominates the tree-based models even though it is a deterministic linear combination of three other features. The importance ranks come out as

Table 2: Top-5 feature importances for the four tree-ensemble classifiers.

Rank	RF	GBT-CV	ExtraTrees	RF-no-v2
1	`v2_score` (0.19)	`v2_score` (0.42)	`v2_score` (0.22)	`artist_score` (0.19)
2	`artist_score`	`user_n_log`	`artist_score`	`has_artist`
3	`artist_above_mean`	`has_album`	`has_artist`	`artist_above_mean`
4	`has_artist`	`genre_above_mean`	`artist_above_mean`	`user_n_log`
5	`has_album`	`art_nrat_log`	`user_n_log`	`has_album`

The Logistic Regression model tells the same story from the linear side: `has_album` (+1.45) and `has_artist` (+1.32) are the two largest coefficients by magnitude, followed by `user_n_log` (−0.58) and `cf_count_log` (+0.53). The negative sign on `user_n_log` is consistent across LR, GBT, and the trees: users with a lot of rating history are harder to predict for, because their candidate tracks are less correlated with their established preferences.

The RF-without-`v2_score` run confirmed that the $v2$ feature is largely redundant: with $v2$ removed, RF redistributed its importance evenly across `artist_score`, `has_artist`, `artist_above_mean`, `has_album`, `album_artist_int`, and `album_score`. The validation AUC was actually 0.0004 *higher* without $v2$, because the model could no longer take a shortcut through the pre-aggregated feature.

4 Ensembles

Six ensemble blends were constructed from the seven individual classifiers' predicted probabilities $P(\text{label} = 1)$:

- `blend_rf_gbt` — equal-weight RF + GBT-CV.
- `blend_rf_heavy` — 2× RF + 1× GBT-CV.
- `blend_three` — equal-weight RF + GBT-CV + LR.
- `blend_trees` — equal-weight RF + GBT-CV + ExtraTrees + RF-no-v2.
- `blend_mega` — AUC-weighted average of all seven classifiers.
- `blend_rank` — average of within-user ranks (rather than raw probabilities) across the four tree models, which is robust to probability calibration differences across families.

5 Kaggle Results

The script generates a top-3-of-6 submission for every individual classifier and every blend. Submissions submitted to Kaggle are summarized below.

Table 3: Kaggle public scores for HW9 submissions (higher is better). Only models actually submitted are listed.

Submission	Val AUC	Kaggle score
`submission_hw9_dt.csv`	0.7899	0.890
`submission_hw9_lr.csv`	0.9376	0.903
`submission_hw9_gbt.csv`	0.9377	0.906
`submission_hw9_gbt_cv.csv`	0.9454	0.909
`submission_hw9_blend_three.csv`	—	0.913
`submission_hw9_rf.csv`	0.9468	**0.914**
`submission_hw9_rf_no_v2.csv`	**0.9472**	**0.914**
`submission_hw9_blend_rank.csv`	—	**0.914**

Validation AUC vs. Kaggle ranking. The validation AUC is a lossy proxy for the Kaggle metric. Validation AUC measures *point-wise* binary classification quality on the 30% held-out portion of the labeled set, while Kaggle measures *within-user, top-3-of-6* ranking accuracy. The first RF run scored val AUC = 0.9472 and Kaggle = 0.908; a second RF run with a different CV grid scored a slightly worse val AUC (0.9468) but a better Kaggle (0.912, then 0.914 after refitting on the full set). The second-place finisher on val AUC (RF-no-`v2`) tied the leader on Kaggle. Probability calibration and within-group ranking sharpness, not raw AUC, are what move the Kaggle score in the third decimal.

Pairwise disagreement among the three 0.914 **winners.** The three models that all hit 0.914 disagree on only 2,697 of 120,000 rows (2.25%), confined to 1,312 of 20,000 users (6.6%). The disagreement pattern is symmetric: each model has roughly 500–600 rows where it is the lone dissenter, with no model consistently outvoting the others. This indicates the three are sampling slightly different points on the same Pareto frontier rather than capturing different underlying signals—adding more tree-family classifiers to this trio is unlikely to produce an additional Kaggle gain.

The `blend_three` attempt. The fourth submission (`blend_three`, equal-weight RF + GBT-CV + LR) was an attempt to inject a different model family's signal into the trio. It scored 0.913, regressing −0.001 from 0.914. The interpretation is that LR's contribution to the blend (1/3 of the weight) carries enough error mass to drag the trio's consensus on a

small number of rows; a smaller LR weight (e.g., 1/8) would likely have closed the gap, but Kaggle's daily submission limit prevented further tuning.

Table 4: Chronological Kaggle submission log for HW9. Three batches of submissions tested progressively stronger configurations.

Batch	Submission	Kaggle score
Batch 1 (initial run, small CV grid)	`submission_hw9_dt.csv`	0.890
	`submission_hw9_gbt.csv`	0.905
	`submission_hw9_lr.csv`	0.905
	`submission_hw9_gbt_cv.csv`	0.907
	`submission_hw9_rf.csv`	0.908
Batch 2 (dropped 3 dead features, bigger CV grid)	`submission_hw9_dt.csv`	0.890
	`submission_hw9_lr.csv`	0.903
	`submission_hw9_gbt.csv`	0.906
	`submission_hw9_gbt_cv.csv`	0.909
	`submission_hw9_rf.csv`	0.912
Batch 3 (refit on full 6,000 rows; ensembles)	`submission_hw9_blend_three.csv`	0.913
	`submission_hw9_rf.csv`	**0.914**
	`submission_hw9_rf_no_v2.csv`	**0.914**
	`submission_hw9_blend_rank.csv`	**0.914**
	`submission_hw9_blend_rf_heavy.csv`	**0.914**

The progression 0.908 → 0.912 → 0.914 on the same Random Forest classifier illustrates the cumulative effect of each engineering change: the second batch dropped three zero-importance features and broadened the CV grid, the third batch refit on the full 6,000 labeled rows after CV completed.

submission_hw9_blend_rf_heavy.csv Complete · 4h ago	0.914	☐
submission_hw9_blend_three.csv Complete · 4h ago	0.913	☐
submission_hw9_rf.csv Complete · 4h ago	0.914	☑
submission_hw9_rf_no_v2.csv Complete · 4h ago	0.914	☐
submission_hw9_blend_rank.csv Complete · 4h ago	0.914	☑
submission_hw9_lr.csv Complete · 5h ago	0.903	☐
submission_hw9_dt.csv Complete · 5h ago	0.890	☐
submission_hw9_rf.csv Complete · 5h ago	0.912	☐
submission_hw9_gbt.csv Complete · 5h ago	0.906	☐
submission_hw9_gbt_cv.csv Complete · 5h ago	0.909	☐
submission_hw9_rf.csv Complete · 5h ago	0.908	☐
submission_hw9_lr.csv Complete · 5h ago	0.905	☐
submission_hw9_gbt_cv.csv Complete · 5h ago	0.907	☐
submission_hw9_gbt.csv Complete · 5h ago	0.905	☐
submission_hw9_dt.csv Complete · 5h ago	0.890	☐

Figure 1: Kaggle public-leaderboard screenshot for the HW9 submissions.

6 Discussion

6.1 Why Tree Ensembles Win

The same observation that recurred from HW6 onward—that pointwise rating regression is a poor fit for a top-k recommendation task—is the reason logistic regression underperforms by ~ 0.01 AUC and the lone Decision Tree underperforms by 0.16 AUC. The signal in this dataset is non-linear in the per-pair features (album rating, artist rating, genre statistics, popularity), and only an ensemble of high-variance learners has the smoothing power to convert that signal into a well-calibrated probability surface. RF works particularly well here because the within-group ranking task only needs the model to sort six candidates correctly, which depends on the *order* of the predicted probabilities, not their absolute values. RF's bagged-tree averaging gives a much smoother probability surface than a single deep tree or a linear model, which is why a 0.79-AUC decision tree turns into a 0.95-AUC random forest.

6.2 Why `v2_score` Dominates and Why It Doesn't Need To

The v2 heuristic was hand-engineered during the midterm to score 0.871 on Kaggle. When fed to a tree as a single feature, it serves as a near-perfect first split: any tree learner instantly captures a large fraction of its predictive power. The CV-tuned GBT puts 42% of its importance on this one feature; the base GBT puts 48%. RF—which has bagging-induced diversity to spend—splits the same signal across many features, but `v2_score` still sits at the top. The key finding is that this dominance is structural, not informational: RF without `v2_score` reaches the same val AUC (slightly higher, in fact) by recovering the same signal from `artist_score`, `has_artist`, and `album_score` instead. The $v2$ feature is a useful shortcut, not a unique signal source.

6.3 Practical Lessons for the Final

Three things from HW9 directly inform the final-project ensemble strategy:

1. **Validation AUC cannot be the only selector.** The model with the best val AUC (RF-no-`v2`) tied for first on Kaggle, but the second- and third-best val-AUC models (RF, GBT-CV) ranged from 0.909 to 0.914. A within-user pair-AUC metric on a held-out group of 6 candidates is a much closer surrogate for the Kaggle metric than per-row binary AUC.

2. **Refitting on the full labeled set after CV is free improvement.** The RF score went from 0.912 (on 70% of the labels) to 0.914 (after refitting on all 6,000). The same trick will be applied to the final-project meta-learner.

3. **Tree ensembles plateau without a different model family.** Three tree-based submissions all scored 0.914 and disagreed on only 2.25% of rows—adding a fourth tree variant won't help. The final's stacked meta-learner therefore consumes nine signal sources spanning four model families: the BPR neural ranker (PyTorch), the v2 heuristic, the seven HW9 PySpark classifiers, and 34 archived past submissions from earlier homework iterations.

7 Conclusion

1. Five PySpark.ML classifiers were trained on the 6,000 labeled rows in `test2_new.txt`, then applied to the 120,000-row final-project Kaggle test set. Random Forest with cross-validated hyperparameters was the strongest single classifier (0.914 Kaggle), tied with an RF variant trained without the redundant `v2_score` feature, and tied with a rank-averaged ensemble of four tree models.

2. The 0.914 result improves on the midterm's best of 0.911 by +0.003 Kaggle points, and importantly does so without any GPU training—the entire pipeline runs in PySpark on CPU.

3. Validation AUC and Kaggle score are correlated but not equal. The model that won on validation AUC also tied on Kaggle, but smaller second-decimal differences in val AUC do not translate cleanly to Kaggle order. Within-group ranking quality (which Kaggle rewards) is a function of probability calibration, not just classification accuracy.

4. Three tree-based submissions all reached 0.914 but disagree on only 2.25% of rows. The plateau suggests that additional gains require either (a) a different model family, or (b) a stacked meta-learner trained on `test2_new.txt` labels using all available score signals. Both are pursued in the final project.

8 Appendix: Python Code

```python
"""
EE627A Homework 9 - PySpark ML Classifiers for Music Recommendation
Jude Eschete

Approach
--------
Feature matrix: 28 hand-engineered features per (user, track) pair,
ported from the midterm v5 model (EE627_Midterm_v5_full.py).
The MF embedding feature is omitted (requires GPU training); all other
29 base features are retained minus that one slot = 28 features.

All features are purely numerical, so no StringIndexer / one-hot
encoding is needed -- we go straight to VectorAssembler then classifiers.

Ground truth: test2_new.txt (1000 users x 6 tracks = 6000 labeled rows).
Prediction target: full testItem2.txt test set (Kaggle submission).

Four PySpark classifiers (Week 12 slides):
  1. Logistic Regression
  2. Decision Tree Classifier
  3. Random Forest Classifier
  4. Gradient-Boosted Tree Classifier  (+ cross-validated tuned variant)

Each generates a Kaggle-format submission CSV.
All output is tee'd to results_hw9.txt for post-run analysis.
"""

import os
import sys
import math
import csv
import time
import statistics
from collections import defaultdict

# ---------------------------------------------------------------------------
# Paths
# ---------------------------------------------------------------------------
SCRIPT_DIR  = os.path.dirname(os.path.abspath(__file__))
MIDTERM_DIR = os.path.join(SCRIPT_DIR, "..", "Midterm")
DATA_DIR    = os.path.join(MIDTERM_DIR, "Data", "music-recommender-2026s")

TRAIN_FILE  = os.path.join(DATA_DIR, "trainItem2.txt")
TEST_FILE   = os.path.join(DATA_DIR, "testItem2.txt")
TRACK_FILE  = os.path.join(DATA_DIR, "trackData2.txt")
ALBUM_FILE  = os.path.join(DATA_DIR, "albumData2.txt")
LABEL_FILE  = os.path.join(SCRIPT_DIR, "test2_new.txt")

FEATURE_CSV  = os.path.join(SCRIPT_DIR, "features_labeled.csv")
TEST_CSV     = os.path.join(SCRIPT_DIR, "features_test.csv")
RESULTS_FILE = os.path.join(SCRIPT_DIR, "results_hw9.txt")

FEATURE_NAMES = [
    "album_score", "has_album",
    "artist_score", "has_artist",
    "genre_count", "genre_max", "genre_min", "genre_mean", "genre_var",
```

```python
    # "evidence_count",       # 0.000 importance on DT, GBT, GBT-CV -- dropped
    "album_artist_int",
    # "genre_artist_int",     # 0.000 on DT, GBT-CV -- dropped
    # "album_genre_int",      # 0.000 on DT, GBT, GBT-CV -- dropped
    "user_mean", "user_std", "user_n_log",
    "album_above_mean", "artist_above_mean", "genre_above_mean",
    "alb_global_avg", "art_global_avg", "genre_global_avg",
    "alb_nrat_log", "art_nrat_log", "genre_nrat_log",
    "cf_score", "cf_count_log",
    "v2_score",
]


# ---------------------------------------------------------------------------
# Logging: tee all print() output to file
# ---------------------------------------------------------------------------

class Tee:
    def __init__(self, filepath):
        self.file   = open(filepath, "w", encoding="utf-8")
        self.stdout = sys.stdout

    def write(self, data):
        self.stdout.write(data)
        self.file.write(data)

    def flush(self):
        self.stdout.flush()
        self.file.flush()

    def close(self):
        self.file.close()
        sys.stdout = self.stdout


def section(title):
    print("\n" + "=" * 65)
    print(f"  {title}")
    print("=" * 65)


def elapsed(t0):
    s = time.time() - t0
    return f"{s:.1f}s" if s < 60 else f"{s/60:.1f}m"


# ---------------------------------------------------------------------------
# 1. Data Parsing
# ---------------------------------------------------------------------------

def parse_training(path):
    t0 = time.time()
    print("Loading training data ...")
    users = {}
    with open(path, "r") as f:
        uid = None
        for line in f:
            line = line.strip()
            if "|" in line:
```

```
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])

    total_ratings = sum(len(r) for r in users.values())
    all_counts    = [len(r) for r in users.values()]
    all_ratings   = [r for u in users.values() for r in u.values()]

    print(f"  {len(users):,} users  |  {total_ratings:,} ratings  [{elapsed(t0)}]")
    print(f"  Ratings per user : min={min(all_counts)}  "
          f"median={sorted(all_counts)[len(all_counts)//2]}  "
          f"max={max(all_counts)}")
    print(f"  Rating values    : min={min(all_ratings)}  "
          f"mean={sum(all_ratings)/len(all_ratings):.2f}  "
          f"max={max(all_ratings)}")
    return users


def parse_tracks(path):
    t0 = time.time()
    print("Loading track metadata ...")
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:] if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)

    no_album  = sum(1 for a, _, _ in tracks.values() if a is None)
    no_artist = sum(1 for _, a, _ in tracks.values() if a is None)
    genre_lens = [len(g) for _, _, g in tracks.values()]
    print(f"  {len(tracks):,} tracks  [{elapsed(t0)}]")
    print(f"  Missing album : {no_album:,}  |  missing artist: {no_artist:,}")
    print(f"  Genres per track: min={min(genre_lens)}  "
          f"mean={sum(genre_lens)/len(genre_lens):.1f}  "
          f"max={max(genre_lens)}")
    return tracks


def parse_albums(path):
    t0 = time.time()
    print("Loading album metadata ...")
    albums = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 1:
                continue
            alb_id = int(parts[0])
            art    = None if len(parts) < 2 or parts[1] == "None" \
                     else int(parts[1])
            genres = [int(g) for g in parts[2:]
```

```
                        if g.strip() and g != "None"]
            albums[alb_id] = (art, genres)
    print(f"  {len(albums):,} albums  [{elapsed(t0)}]")
    return albums


def parse_test(path):
    t0 = time.time()
    print("Loading test candidates ...")
    test = []
    with open(path, "r") as f:
        uid  = None
        tids = []
        for line in f:
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid  = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
            test.append((uid, tids))

    n_cands = [len(t) for _, t in test]
    print(f"  {len(test):,} test users  |  "
          f"{sum(n_cands):,} total candidates  [{elapsed(t0)}]")
    print(f"  Candidates per user: "
          f"min={min(n_cands)}  max={max(n_cands)}  "
          f"(expected 6 each)")
    return test


def parse_labels(path):
    t0 = time.time()
    print("Loading ground-truth labels ...")
    labels = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) == 3:
                labels[(int(parts[0]), int(parts[1]))] = int(parts[2])

    n_pos = sum(v for v in labels.values())
    n_neg = len(labels) - n_pos
    uids  = set(u for u, _ in labels)
    print(f"  {len(labels):,} rows  |  {len(uids):,} users  [{elapsed(t0)}]")
    print(f"  Label balance: {n_pos:,} positives ({100*n_pos/len(labels):.1f}%)  "
          f"|  {n_neg:,} negatives ({100*n_neg/len(labels):.1f}%)")
    return labels


# ---------------------------------------------------------------------------
# 2. Enrichment
# ---------------------------------------------------------------------------

def enrich_tracks(tracks, albums):
    t0 = time.time()
```

```python
    print("Enriching tracks with album metadata ...")
    enriched       = {}
    artist_filled  = 0
    genres_added   = 0
    for tid, (alb, art, genres) in tracks.items():
        new_art    = art
        new_genres = list(genres)
        if alb is not None and alb in albums:
            album_art, album_genres = albums[alb]
            if art is None and album_art is not None:
                new_art = album_art
                artist_filled += 1
            seen = set(genres)
            for g in album_genres:
                if g not in seen:
                    new_genres.append(g)
                    seen.add(g)
                    genres_added += 1
        enriched[tid] = (alb, new_art, new_genres)

    no_artist_after = sum(1 for _, a, _ in enriched.values() if a is None)
    print(f"  Artists filled via album : {artist_filled:,}")
    print(f"  Genres added via album   : {genres_added:,}")
    print(f"  Tracks still missing artist after enrichment: "
          f"{no_artist_after:,}  [{elapsed(t0)}]")
    return enriched


# ---------------------------------------------------------------------------
# 3. Auxiliary structures
# ---------------------------------------------------------------------------

def build_global_item_stats(user_ratings):
    t0       = time.time()
    item_sum = defaultdict(float)
    item_cnt = defaultdict(int)
    for ratings in user_ratings.values():
        for iid, r in ratings.items():
            item_sum[iid] += r
            item_cnt[iid] += 1
    stats = {iid: (item_sum[iid] / item_cnt[iid], item_cnt[iid])
             for iid in item_sum}

    counts = [c for _, c in stats.values()]
    avgs   = [a for a, _ in stats.values()]
    print(f"  Global item stats: {len(stats):,} items  [{elapsed(t0)}]")
    print(f"  Raters per item  : min={min(counts)}  "
          f"median={sorted(counts)[len(counts)//2]}  "
          f"max={max(counts)}")
    print(f"  Global avg rating: min={min(avgs):.1f}  "
          f"mean={sum(avgs)/len(avgs):.2f}  "
          f"max={max(avgs):.1f}")
    return stats


def build_track_mappings(track_meta):
    album_to_tracks  = defaultdict(list)
    artist_to_tracks = defaultdict(list)
    for tid, (alb, art, genres) in track_meta.items():
```

```python
        if alb is not None:
            album_to_tracks[alb].append(tid)
        if art is not None:
            artist_to_tracks[art].append(tid)
    alb_sizes = [len(v) for v in album_to_tracks.values()]
    art_sizes = [len(v) for v in artist_to_tracks.values()]
    print(f"  Album->tracks  : {len(album_to_tracks):,} albums  "
          f"(avg {sum(alb_sizes)/len(alb_sizes):.1f} tracks/album)")
    print(f"  Artist->tracks : {len(artist_to_tracks):,} artists  "
          f"(avg {sum(art_sizes)/len(art_sizes):.1f} tracks/artist)")
    return dict(album_to_tracks), dict(artist_to_tracks)


# ---------------------------------------------------------------------------
# 4. Feature computation (28 features -- v5 base minus MF slot)
# ---------------------------------------------------------------------------

def compute_user_profile(ratings):
    vals = list(ratings.values())
    n    = len(vals)
    if n == 0:
        return 0.0, 0.0, 0
    mean = sum(vals) / n
    std  = (sum((v - mean) ** 2 for v in vals) / n) ** 0.5 if n > 1 else 0.0
    return mean, std, n


def compute_features(uid, tid, ratings, track_meta, global_stats,
                     album_to_tracks, artist_to_tracks):
    """28 numerical features per (user, track) pair.

    Matches v5 base features 0-26 and 28 (v2_score).
    Feature 27 (MF dot product) is omitted -- requires GPU MF training.
    """
    alb_id, art_id, genre_ids = track_meta.get(tid, (None, None, []))
    u_mean, u_std, u_n        = compute_user_profile(ratings)

    # Album
    if alb_id is not None and alb_id in ratings:
        album_score, has_album = float(ratings[alb_id]), 1.0
    else:
        album_score, has_album = 0.0, 0.0

    # Artist
    if art_id is not None and art_id in ratings:
        artist_score, has_artist = float(ratings[art_id]), 1.0
    else:
        artist_score, has_artist = 0.0, 0.0

    # Genre stats
    genre_scores = [ratings[g] for g in genre_ids if g in ratings]
    gc = len(genre_scores)
    if gc > 0:
        g_max  = float(max(genre_scores))
        g_min  = float(min(genre_scores))
        g_mean = sum(genre_scores) / gc
        g_var  = sum((s - g_mean) ** 2 for s in genre_scores) / gc
    else:
        g_max = g_min = g_mean = g_var = 0.0
```

```python

    evidence_count = has_album + has_artist + (1.0 if gc > 0 else 0.0)

    # Interaction terms
    aa_int = album_score * artist_score / 100.0 \
             if (has_album and has_artist) else 0.0
    ga_int = g_mean * artist_score / 100.0 \
             if (has_artist and gc > 0) else 0.0
    ag_int = album_score * g_mean / 100.0 \
             if (has_album and gc > 0) else 0.0

    # User-relative scores
    album_above  = (album_score  - u_mean) if has_album  else 0.0
    artist_above = (artist_score - u_mean) if has_artist else 0.0
    genre_above  = (g_mean       - u_mean) if gc > 0     else 0.0

    # Global popularity stats (log-scaled to suppress heavy-tail outliers)
    alb_gavg, alb_nrat = global_stats.get(alb_id, (0.0, 0)) \
                         if alb_id else (0.0, 0)
    art_gavg, art_nrat = global_stats.get(art_id, (0.0, 0)) \
                         if art_id else (0.0, 0)
    g_gavg_sum = g_nrat_sum = g_items = 0.0
    for gid in genre_ids:
        if gid in global_stats:
            gavg, nrat  = global_stats[gid]
            g_gavg_sum += gavg
            g_nrat_sum += nrat
            g_items    += 1
    avg_genre_gavg = g_gavg_sum / g_items if g_items > 0 else 0.0
    avg_genre_nrat = g_nrat_sum / g_items if g_items > 0 else 0.0

    # CF sibling features
    sibling_ratings = []
    if alb_id is not None:
        for sib in album_to_tracks.get(alb_id, []):
            if sib != tid and sib in ratings:
                sibling_ratings.append(ratings[sib])
    if art_id is not None:
        album_sibs = set(album_to_tracks.get(alb_id, [])) if alb_id else set()
        for sib in artist_to_tracks.get(art_id, []):
            if sib != tid and sib not in album_sibs and sib in ratings:
                sibling_ratings.append(ratings[sib])
    cf_score = sum(sibling_ratings) / len(sibling_ratings) \
               if sibling_ratings else 0.0
    cf_count = len(sibling_ratings)

    # v2 heuristic score (most informative single signal, Kaggle-validated)
    v2_score = (has_album  * album_score
                + has_artist * artist_score
                + gc * g_mean / 10.0)

    return [
        album_score,   has_album,                       # 0-1
        artist_score,  has_artist,                      # 2-3
        float(gc), g_max, g_min, g_mean, g_var,         # 4-8
        evidence_count,                                 # 9
        aa_int, ga_int, ag_int,                         # 10-12
        u_mean, u_std, math.log1p(u_n),                 # 13-15
        album_above, artist_above, genre_above,         # 16-18
```

```
        alb_gavg,  art_gavg,  avg_genre_gavg,            # 19-21
        math.log1p(alb_nrat), math.log1p(art_nrat),      # 22-23
        math.log1p(avg_genre_nrat),                      # 24
        cf_score, math.log1p(cf_count),                  # 25-26
        v2_score,                                        # 27
    ]


# ---------------------------------------------------------------------------
# 5. Build feature CSVs + inline diagnostics
# ---------------------------------------------------------------------------

def build_labeled_csv(user_ratings, track_meta, global_stats,
                      album_to_tracks, artist_to_tracks, labels, out_path):
    t0 = time.time()
    print(f"Building labeled feature CSV ...")
    header = ["user_id", "track_id"] + FEATURE_NAMES + ["label"]

    all_feats  = [[] for _ in FEATURE_NAMES]  # per-feature value lists
    rows_written = 0
    missing_user = 0

    with open(out_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(header)
        for (uid, tid), label in labels.items():
            ratings = user_ratings.get(uid, {})
            if not ratings:
                missing_user += 1
            feats = compute_features(uid, tid, ratings, track_meta,
                                     global_stats, album_to_tracks,
                                     artist_to_tracks)
            writer.writerow([uid, tid] + feats + [label])
            for i, v in enumerate(feats):
                all_feats[i].append(v)
            rows_written += 1

    print(f"  {rows_written:,} rows written  "
          f"|  {missing_user:,} users with no training history  [{elapsed(t0)}]")

    # Feature distribution diagnostics
    print("\n  Feature distributions (labeled set):")
    print(f"  {'Feature':<25} {'mean':>8} {'std':>8} "
          f"{'min':>8} {'max':>8} {'zero%':>7}")
    print(f"  {'-'*25} {'-'*8} {'-'*8} {'-'*8} {'-'*8} {'-'*7}")
    for i, name in enumerate(FEATURE_NAMES):
        vals  = all_feats[i]
        mean  = sum(vals) / len(vals)
        std   = statistics.pstdev(vals)
        vmin  = min(vals)
        vmax  = max(vals)
        zpct  = 100.0 * sum(1 for v in vals if v == 0.0) / len(vals)
        flag  = "  <- mostly zero, consider dropping" if zpct > 90 else ""
        print(f"  {name:<25} {mean:>8.2f} {std:>8.2f} "
              f"{vmin:>8.2f} {vmax:>8.2f} {zpct:>6.1f}%{flag}")


def build_test_csv(user_ratings, track_meta, global_stats,
                   album_to_tracks, artist_to_tracks, test_users, out_path):
```

```python
    t0 = time.time()
    print(f"Building test feature CSV ...")
    header       = ["user_id", "track_id"] + FEATURE_NAMES
    rows_written = 0
    missing_user = 0
    with open(out_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(header)
        for uid, candidates in test_users:
            ratings = user_ratings.get(uid, {})
            if not ratings:
                missing_user += 1
            for tid in candidates:
                feats = compute_features(uid, tid, ratings, track_meta,
                                         global_stats, album_to_tracks,
                                         artist_to_tracks)
                writer.writerow([uid, tid] + feats)
                rows_written += 1
    print(f"  {rows_written:,} test rows written  "
          f"|  {missing_user:,} users with no training history  [{elapsed(t0)}]")


# ---------------------------------------------------------------------------
# 6. PySpark ML pipeline
# ---------------------------------------------------------------------------

def log_classifier_metrics(name, preds_val, evaluator, train_df):
    """Print AUC, precision, recall, confusion matrix on validation set."""
    from pyspark.ml.evaluation import MulticlassClassificationEvaluator

    auc = evaluator.evaluate(preds_val,
                             {evaluator.metricName: "areaUnderROC"})
    mc  = MulticlassClassificationEvaluator(labelCol="label",
                                            predictionCol="prediction")
    acc  = mc.evaluate(preds_val, {mc.metricName: "accuracy"})
    prec = mc.evaluate(preds_val, {mc.metricName: "weightedPrecision"})
    rec  = mc.evaluate(preds_val, {mc.metricName: "weightedRecall"})
    f1   = mc.evaluate(preds_val, {mc.metricName: "f1"})

    # Confusion matrix counts
    pdf = preds_val.select("label", "prediction").toPandas()
    tp = int(((pdf["label"] == 1) & (pdf["prediction"] == 1)).sum())
    tn = int(((pdf["label"] == 0) & (pdf["prediction"] == 0)).sum())
    fp = int(((pdf["label"] == 0) & (pdf["prediction"] == 1)).sum())
    fn = int(((pdf["label"] == 1) & (pdf["prediction"] == 0)).sum())

    print(f"\n  [{name}] Validation metrics:")
    print(f"    AUC-ROC   : {auc:.4f}")
    print(f"    Accuracy  : {acc:.4f}")
    print(f"    Precision : {prec:.4f}")
    print(f"    Recall    : {rec:.4f}")
    print(f"    F1        : {f1:.4f}")
    print(f"    Confusion matrix (val set):")
    print(f"      Predicted ->     0       1")
    print(f"      Actual 0    {tn:>6,}  {fp:>6,}   (spec={tn/(tn+fp):.3f})")
    print(f"      Actual 1    {fn:>6,}  {tp:>6,}   (sens={tp/(tp+fn):.3f})")
    return auc


```

```python
def log_feature_importances(feat_importances, label, feature_names=None):
    """Print full ranked feature importance table."""
    if feature_names is None:
        feature_names = FEATURE_NAMES
    arr    = feat_importances.toArray()
    ranked = sorted(enumerate(arr), key=lambda x: x[1], reverse=True)
    print(f"\n  [{label}] Feature importances (all {len(arr)} features):")
    print(f"  {'Rank':<5} {'Feature':<25} {'Importance':>12}  Bar")
    print(f"  {'-'*5} {'-'*25} {'-'*12}  {'-'*20}")
    max_imp = ranked[0][1] if ranked else 1.0
    for rank, (idx, imp) in enumerate(ranked, 1):
        bar  = "#" * int(20 * imp / max_imp)
        flag = "  <- low signal" if imp < 0.005 else ""
        print(f"  {rank:<5} {feature_names[idx]:<25} {imp:>12.6f}  {bar}{flag}")


def log_cv_fold_scores(cv_model):
    """Print per-fold AUC from cross-validation."""
    avg_metrics = cv_model.avgMetrics
    print(f"\n  [GBT-CV] Per-parameter-combination average AUC "
          f"({len(avg_metrics)} combinations):")
    for i, score in enumerate(avg_metrics):
        print(f"    Combo {i+1:2d}: {score:.4f}")
    best_idx = avg_metrics.index(max(avg_metrics))
    print(f"  Best combination index: {best_idx+1}  "
          f"(AUC = {avg_metrics[best_idx]:.4f})")


def collect_probs(pred_df):
    """Collect (uid, tid, prob1) tuples from a Spark prediction DataFrame.
    Returns dict {(uid, tid): prob1}."""
    rows = pred_df.select("user_id", "track_id", "probability").collect()
    return {(int(r["user_id"]), int(r["track_id"])): float(r["probability"][1])
            for r in rows}


def write_blend_submission(blend_dict, name, script_dir, label):
    """Take a {(uid, tid): score} dict and write a top-3-per-user CSV.
    `label` is a short string used in the printed summary."""
    by_user = defaultdict(list)
    for (uid, tid), s in blend_dict.items():
        by_user[uid].append((tid, s))

    sub_path = os.path.join(script_dir, f"submission_hw9_{name}.csv")
    n_ones, n_total = 0, 0
    with open(sub_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["TrackID", "Predictor"])
        for uid, items in by_user.items():
            top3 = set(t for t, _ in sorted(items, key=lambda x: x[1],
                                            reverse=True)[:3])
            for tid, _ in items:
                pred = 1 if tid in top3 else 0
                writer.writerow([f"{uid}_{tid}", pred])
                n_ones  += pred
                n_total += 1
    print(f"  {name:<22} ({label})  "
          f"{n_ones:,} rec / {n_total-n_ones:,} not rec  "
          f"-> {os.path.basename(sub_path)}")
```

```python
    return sub_path


def run_pyspark(feature_csv, test_csv, script_dir):
    t_spark = time.time()

    from pyspark.sql import SparkSession
    from pyspark.sql.functions import col
    from pyspark.ml.feature import VectorAssembler, StandardScaler
    from pyspark.ml import Pipeline
    from pyspark.ml.classification import (LogisticRegression,
                                           DecisionTreeClassifier,
                                           RandomForestClassifier,
                                           GBTClassifier)
    from pyspark.ml.evaluation import BinaryClassificationEvaluator
    from pyspark.ml.tuning import ParamGridBuilder, CrossValidator

    # ---- SparkSession ----
    # Bump thread stack to 32M to handle deep RF/GBT model serialization.
    # Default 1M overflows when deserializing trees with maxDepth >= ~15.
    spark = (SparkSession.builder
             .appName("EE627-HW9")
             .config("spark.driver.memory",          "6g")
             .config("spark.driver.extraJavaOptions", "-Xss32m")
             .config("spark.executor.extraJavaOptions","-Xss32m")
             .getOrCreate())
    spark.sparkContext.setLogLevel("WARN")

    section("LOADING & PREPARING DATA IN SPARK")

    labeled = spark.read.csv(feature_csv, header=True, inferSchema=True)
    labeled = labeled.withColumn("label", col("label").cast("double"))
    n_total = labeled.count()
    n_pos   = labeled.filter(col("label") == 1).count()
    n_neg   = n_total - n_pos
    print(f"  Labeled rows : {n_total:,}  "
          f"({n_pos:,} pos = {100*n_pos/n_total:.1f}%  "
          f"|  {n_neg:,} neg = {100*n_neg/n_total:.1f}%)")

    assembler   = VectorAssembler(inputCols=FEATURE_NAMES, outputCol="features")
    prep_model  = Pipeline(stages=[assembler]).fit(labeled)
    df_full     = prep_model.transform(labeled) \
                          .select("label", "features", "user_id", "track_id")

    # 70/30 split for hyperparameter selection / validation reporting only
    train, val = df_full.randomSplit([0.7, 0.3], seed=2018)
    n_train    = train.count()
    n_val      = val.count()
    n_val_pos  = val.filter(col("label") == 1).count()
    print(f"  Full labeled : {n_total:,}  (used to refit final models)")
    print(f"  Train (70%)  : {n_train:,} rows  (used for CV & tuning)")
    print(f"  Validation   : {n_val:,} rows  "
          f"({n_val_pos:,} pos = {100*n_val_pos/n_val:.1f}%)")

    # Load full Kaggle test set
    test_df_raw = spark.read.csv(test_csv, header=True, inferSchema=True)
    test_df     = prep_model.transform(test_df_raw) \
                          .select("user_id", "track_id", "features")
    print(f"  Kaggle test  : {test_df.count():,} rows")
```

```python

    evaluator = BinaryClassificationEvaluator(metricName="areaUnderROC")
    results   = {}   # name -> (auc, prob_dict)  prob_dict = {(uid,tid): p}

    # ----------------------------------------------------------------
    # 1. Logistic Regression  (elastic net + light tuning)
    # ----------------------------------------------------------------
    section("[1/7] LOGISTIC REGRESSION (elastic-net CV)")
    t0 = time.time()
    lr_tune = LogisticRegression(featuresCol="features", labelCol="label",
                                 maxIter=50)
    lr_grid = (ParamGridBuilder()
               .addGrid(lr_tune.regParam,        [0.0, 0.01, 0.1])
               .addGrid(lr_tune.elasticNetParam, [0.0, 0.5, 1.0])
               .build())
    print(f"  Grid size: {len(lr_grid)} combos x 3 folds = {len(lr_grid)*3} fits")
    lr_cv    = CrossValidator(estimator=lr_tune,
                              estimatorParamMaps=lr_grid,
                              evaluator=evaluator,
                              numFolds=3,
                              seed=42)
    lr_cv_model = lr_cv.fit(train)
    lr_best     = lr_cv_model.bestModel
    lr_val      = lr_cv_model.transform(val)
    lr_auc      = log_classifier_metrics("LR", lr_val, evaluator, train)

    print(f"\n  [LR-CV] Best hyperparameters:")
    print(f"    regParam        = {lr_best.getRegParam()}")
    print(f"    elasticNetParam = {lr_best.getElasticNetParam()}")

    coeffs   = lr_best.coefficients.toArray()
    ranked_c = sorted(enumerate(coeffs), key=lambda x: abs(x[1]), reverse=True)
    print(f"\n  [LR] Top-10 coefficients (|beta|):")
    print(f"  {'Rank':<5} {'Feature':<25} {'Coefficient':>12}")
    print(f"  {'-'*5} {'-'*25} {'-'*12}")
    for rank, (idx, coef) in enumerate(ranked_c[:10], 1):
        sign = "+" if coef >= 0 else ""
        print(f"  {rank:<5} {FEATURE_NAMES[idx]:<25} {sign}{coef:>11.4f}")

    # Refit on full 6,000 rows with best params
    print(f"\n  [LR] Refitting on full {n_total:,} rows with best params...")
    lr_full = LogisticRegression(featuresCol="features", labelCol="label",
                                 maxIter=50,
                                 regParam=lr_best.getRegParam(),
                                 elasticNetParam=lr_best.getElasticNetParam())
    lr_full_model = lr_full.fit(df_full)
    print(f"  LR-CV training time: {elapsed(t0)}")
    results["lr"] = (lr_auc, collect_probs(lr_full_model.transform(test_df)))

    # ----------------------------------------------------------------
    # 2. Decision Tree (kept for HW9 spec, single tree baseline)
    # ----------------------------------------------------------------
    section("[2/7] DECISION TREE CLASSIFIER")
    t0 = time.time()
    dt       = DecisionTreeClassifier(featuresCol="features", labelCol="label",
                                      maxDepth=5)
    dt_model = dt.fit(train)
    dt_val   = dt_model.transform(val)
    dt_auc   = log_classifier_metrics("DT", dt_val, evaluator, train)
```

```
    log_feature_importances(dt_model.featureImportances, "DT")
    print(f"\n  DT depth used : {dt_model.depth}  "
          f"|  nodes: {dt_model.numNodes}")
    # Refit DT on full set
    dt_full = DecisionTreeClassifier(featuresCol="features", labelCol="label",
                                     maxDepth=5).fit(df_full)
    print(f"  DT training time: {elapsed(t0)}")
    results["dt"] = (dt_auc, collect_probs(dt_full.transform(test_df)))

    # ----------------------------------------------------------------
    # 3. Random Forest (BIG CV grid + refit on full set)
    # ----------------------------------------------------------------
    section("[3/7] RANDOM FOREST CLASSIFIER (Big CV-tuned)")
    t0 = time.time()
    rf_tune = RandomForestClassifier(featuresCol="features", labelCol="label",
                                     seed=42)
    # NOTE: maxDepth >= 20 causes StackOverflowError during serialization
    # on Windows JVMs with default -Xss1m. Cap at 15.
    rf_grid = (ParamGridBuilder()
               .addGrid(rf_tune.numTrees,               [200, 500])
               .addGrid(rf_tune.maxDepth,               [10, 12, 15])
               .addGrid(rf_tune.featureSubsetStrategy,  ["sqrt", "log2"])
               .build())
    print(f"  Grid size: {len(rf_grid)} combos x 5 folds = {len(rf_grid)*5} fits")
    rf_cv = CrossValidator(estimator=rf_tune,
                           estimatorParamMaps=rf_grid,
                           evaluator=evaluator,
                           numFolds=5,
                           seed=42)
    rf_cv_model = rf_cv.fit(train)
    rf_best     = rf_cv_model.bestModel
    rf_val      = rf_cv_model.transform(val)
    rf_auc      = log_classifier_metrics("RF", rf_val, evaluator, train)

    print(f"\n  [RF-CV] Best hyperparameters:")
    print(f"    numTrees             = {rf_best.getNumTrees}")
    print(f"    maxDepth             = {rf_best.getMaxDepth()}")
    print(f"    featureSubsetStrategy= {rf_best.getFeatureSubsetStrategy()}")
    log_cv_fold_scores(rf_cv_model)
    log_feature_importances(rf_best.featureImportances, "RF-CV best")

    # Refit on FULL 6,000 rows -- this is the big winner from last run
    print(f"\n  [RF] Refitting on full {n_total:,} rows with best params...")
    rf_full = RandomForestClassifier(
        featuresCol="features", labelCol="label", seed=42,
        numTrees=rf_best.getNumTrees,
        maxDepth=rf_best.getMaxDepth(),
        featureSubsetStrategy=rf_best.getFeatureSubsetStrategy()
    ).fit(df_full)
    print(f"  RF-CV+refit training time: {elapsed(t0)}")
    results["rf"] = (rf_auc, collect_probs(rf_full.transform(test_df)))

    # ----------------------------------------------------------------
    # 4. Gradient-Boosted Tree (base)
    # ----------------------------------------------------------------
    section("[4/7] GRADIENT-BOOSTED TREE (base)")
    t0 = time.time()
    gbt       = GBTClassifier(featuresCol="features", labelCol="label",
                              maxIter=10, stepSize=0.1, seed=42)
```

```
    gbt_model = gbt.fit(train)
    gbt_val   = gbt_model.transform(val)
    gbt_auc   = log_classifier_metrics("GBT", gbt_val, evaluator, train)
    log_feature_importances(gbt_model.featureImportances, "GBT")
    gbt_full = GBTClassifier(featuresCol="features", labelCol="label",
                             maxIter=10, stepSize=0.1, seed=42).fit(df_full)
    print(f"  GBT training time: {elapsed(t0)}")
    results["gbt"] = (gbt_auc, collect_probs(gbt_full.transform(test_df)))

    # ------------------------------------------------------------------
    # 5. GBT + Big Cross-Validated Hyperparameter Tuning + refit
    # ------------------------------------------------------------------
    section("[5/7] GBT + BIG CROSS-VALIDATION TUNING")
    t0       = time.time()
    gbt_tune = GBTClassifier(featuresCol="features", labelCol="label",
                             seed=42)
    param_grid = (ParamGridBuilder()
                  .addGrid(gbt_tune.maxDepth,        [3, 5])
                  .addGrid(gbt_tune.maxIter,         [50, 100, 200])
                  .addGrid(gbt_tune.stepSize,        [0.05, 0.1])
                  .addGrid(gbt_tune.subsamplingRate, [0.7, 1.0])
                  .build())
    print(f"  Grid size: {len(param_grid)} combos x 5 folds "
          f"= {len(param_grid)*5} fits")

    cv = CrossValidator(estimator=gbt_tune,
                        estimatorParamMaps=param_grid,
                        evaluator=evaluator,
                        numFolds=5,
                        seed=42)
    cv_model = cv.fit(train)
    cv_val   = cv_model.transform(val)
    cv_auc   = log_classifier_metrics("GBT-CV", cv_val, evaluator, train)

    best = cv_model.bestModel
    print(f"\n  [GBT-CV] Best hyperparameters:")
    print(f"    maxDepth        = {best.getMaxDepth()}")
    print(f"    maxIter         = {best.getMaxIter()}")
    print(f"    stepSize        = {best.getStepSize()}")
    print(f"    subsamplingRate = {best.getSubsamplingRate()}")
    log_cv_fold_scores(cv_model)
    log_feature_importances(best.featureImportances, "GBT-CV best")

    print(f"\n  [GBT-CV] Refitting on full {n_total:,} rows with best params...")
    gbt_cv_full = GBTClassifier(
        featuresCol="features", labelCol="label", seed=42,
        maxDepth=best.getMaxDepth(),
        maxIter=best.getMaxIter(),
        stepSize=best.getStepSize(),
        subsamplingRate=best.getSubsamplingRate()
    ).fit(df_full)
    print(f"  GBT-CV+refit training time: {elapsed(t0)}")
    results["gbt_cv"] = (cv_auc, collect_probs(gbt_cv_full.transform(test_df)))

    # ------------------------------------------------------------------
    # 6. RF without v2_score (forces model to use raw features)
    # ------------------------------------------------------------------
    section("[6/7] RF WITHOUT v2_score (raw-feature variant)")
    t0 = time.time()
```

```
    no_v2_features = [f for f in FEATURE_NAMES if f != "v2_score"]
    print(f"  Features used: {len(no_v2_features)} (dropped v2_score)")
    asm_nov2 = VectorAssembler(inputCols=no_v2_features, outputCol="features")
    df_nov2  = asm_nov2.transform(labeled).select("label", "features",
                                                  "user_id", "track_id")
    test_nov2 = asm_nov2.transform(test_df_raw).select("user_id", "track_id",
                                                       "features")
    train_nv, val_nv = df_nov2.randomSplit([0.7, 0.3], seed=2018)

    rf_nv = RandomForestClassifier(
        featuresCol="features", labelCol="label", seed=42,
        numTrees=rf_best.getNumTrees,
        maxDepth=rf_best.getMaxDepth(),
        featureSubsetStrategy=rf_best.getFeatureSubsetStrategy()
    )
    rf_nv_model = rf_nv.fit(train_nv)
    rf_nv_val   = rf_nv_model.transform(val_nv)
    rf_nv_auc   = log_classifier_metrics("RF-no-v2", rf_nv_val, evaluator,
                                         train_nv)
    log_feature_importances(rf_nv_model.featureImportances, "RF-no-v2",
                            feature_names=no_v2_features)

    rf_nv_full = RandomForestClassifier(
        featuresCol="features", labelCol="label", seed=42,
        numTrees=rf_best.getNumTrees,
        maxDepth=rf_best.getMaxDepth(),
        featureSubsetStrategy=rf_best.getFeatureSubsetStrategy()
    ).fit(df_nov2)
    print(f"  RF-no-v2 training time: {elapsed(t0)}")
    results["rf_no_v2"] = (rf_nv_auc, collect_probs(rf_nv_full.transform(test_nov2)))

    # ----------------------------------------------------------------
    # 7. Extra Trees -- RF with all-feature selection per split (more random)
    # ----------------------------------------------------------------
    section("[7/7] EXTRA TREES (RF, more randomness)")
    t0 = time.time()
    rf_extra = RandomForestClassifier(
        featuresCol="features", labelCol="label", seed=123,
        numTrees=500,
        maxDepth=15,
        featureSubsetStrategy="onethird",
        subsamplingRate=0.8
    )
    rf_extra_model = rf_extra.fit(train)
    rf_extra_val   = rf_extra_model.transform(val)
    rf_extra_auc   = log_classifier_metrics("ExtraT", rf_extra_val, evaluator,
                                            train)
    log_feature_importances(rf_extra_model.featureImportances, "ExtraT")

    rf_extra_full = RandomForestClassifier(
        featuresCol="features", labelCol="label", seed=123,
        numTrees=500, maxDepth=15,
        featureSubsetStrategy="onethird", subsamplingRate=0.8
    ).fit(df_full)
    print(f"  ExtraT training time: {elapsed(t0)}")
    results["extra"] = (rf_extra_auc,
                        collect_probs(rf_extra_full.transform(test_df)))

    # ----------------------------------------------------------------
```

```python
    # Side-by-side summary
    # ----------------------------------------------------------------
    section("VALIDATION AUC SUMMARY")
    display_names = {
        "lr":       "Logistic Regression CV",
        "dt":       "Decision Tree         ",
        "rf":       "Random Forest (Big CV)",
        "gbt":      "GBT (base)            ",
        "gbt_cv":   "GBT (Big CV)          ",
        "rf_no_v2": "RF without v2_score   ",
        "extra":    "Extra Trees           ",
    }
    sorted_results = sorted(results.items(), key=lambda x: x[1][0], reverse=True)
    best_name, (best_auc, _) = sorted_results[0]
    print(f"  {'Classifier':<26} {'Val AUC':>9}  {'vs best':>8}")
    print(f"  {'-'*26} {'-'*9}  {'-'*8}")
    for name, (auc, _) in sorted_results:
        delta = auc - best_auc
        marker = "  <- BEST" if name == best_name else ""
        print(f"  {display_names[name]:<26} {auc:>9.4f}  {delta:>+8.4f}{marker}")

    # ----------------------------------------------------------------
    # Write per-classifier Kaggle submissions
    # ----------------------------------------------------------------
    section("WRITING PER-CLASSIFIER KAGGLE SUBMISSIONS")
    for name, (auc, probs) in results.items():
        write_blend_submission(probs, name, script_dir,
                               label=f"AUC={auc:.4f}")

    # ----------------------------------------------------------------
    # ENSEMBLE BLENDS  -- where the magic happens
    # ----------------------------------------------------------------
    section("ENSEMBLE BLENDS")

    def blend(weights):
        """weights: dict {classifier_name: weight}.  Returns merged prob dict."""
        keys  = list(results[next(iter(weights))][1].keys())
        out   = {}
        wsum  = sum(weights.values())
        for k in keys:
            score = 0.0
            for name, w in weights.items():
                score += w * results[name][1].get(k, 0.0)
            out[k] = score / wsum
        return out

    # 1. RF + GBT-CV equal blend  (top-2 individual models)
    print(f"\n  Building rf_gbt_blend (RF 0.5 + GBT-CV 0.5)...")
    blend_rf_gbt = blend({"rf": 1.0, "gbt_cv": 1.0})
    write_blend_submission(blend_rf_gbt, "blend_rf_gbt", script_dir,
                           label="RF + GBT-CV equal")

    # 2. RF heavy + GBT-CV light  (RF won on Kaggle -- weight it more)
    print(f"  Building rf_heavy_blend (RF 2 + GBT-CV 1)...")
    blend_rf_heavy = blend({"rf": 2.0, "gbt_cv": 1.0})
    write_blend_submission(blend_rf_heavy, "blend_rf_heavy", script_dir,
                           label="RF 2 + GBT-CV 1")

    # 3. Three-model average: RF + GBT-CV + LR
```

```python
    print(f"  Building three_model_blend (RF + GBT-CV + LR equal)...")
    blend_three = blend({"rf": 1.0, "gbt_cv": 1.0, "lr": 1.0})
    write_blend_submission(blend_three, "blend_three", script_dir,
                           label="RF + GBT-CV + LR")

    # 4. Tree ensemble: RF + GBT-CV + Extra + RF-no-v2 (4 tree variants)
    print(f"  Building tree_ensemble (RF + GBT-CV + Extra + RF-no-v2)...")
    blend_trees = blend({"rf": 1.0, "gbt_cv": 1.0,
                         "extra": 1.0, "rf_no_v2": 1.0})
    write_blend_submission(blend_trees, "blend_trees", script_dir,
                           label="4-tree ensemble")

    # 5. Mega blend: ALL classifiers AUC-weighted
    print(f"  Building mega_blend (all 7 classifiers, AUC-weighted)...")
    auc_weights = {n: a for n, (a, _) in results.items()}
    blend_mega  = blend(auc_weights)
    write_blend_submission(blend_mega, "blend_mega", script_dir,
                           label="ALL, AUC-weighted")

    # 6. Rank-based blend: convert probs to within-user ranks, then average.
    #    This is more robust to probability calibration differences.
    print(f"  Building rank_blend (rank-averaged across models)...")
    rank_blend = rank_average_blend(["rf", "gbt_cv", "extra", "rf_no_v2"],
                                    results)
    write_blend_submission(rank_blend, "blend_rank", script_dir,
                           label="rank-avg of 4 trees")

    # ------------------------------------------------------------------
    # Save all per-(uid,tid) probabilities to disk for Final project use
    # ------------------------------------------------------------------
    section("SAVING PROBABILITY MATRIX FOR FINAL PROJECT")
    prob_matrix_path = os.path.join(script_dir, "hw9_probabilities.csv")
    all_keys = list(results["rf"][1].keys())
    model_order = ["lr", "dt", "rf", "gbt", "gbt_cv", "rf_no_v2", "extra"]
    with open(prob_matrix_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["user_id", "track_id"] + model_order)
        for k in all_keys:
            row = [k[0], k[1]] + [results[m][1].get(k, 0.0) for m in model_order]
            writer.writerow(row)
    print(f"  Wrote {len(all_keys):,} rows x {len(model_order)} models -> "
          f"{os.path.basename(prob_matrix_path)}")
    print(f"  This file is the input matrix for the Final project meta-learner.")

    section("IMPROVEMENT OPPORTUNITIES")
    print("  After this run, the natural next steps are:")
    print("  1. Submit blend_rank, blend_trees, blend_rf_heavy first --")
    print("     these are the highest-EV blends (diverse signal sources).")
    print("  2. Compare blend Kaggle scores to the best single model --")
    print("     if mega_blend > rf alone, ensembling is paying off.")
    print("  3. If RF-no-v2 scored well, it means v2_score was redundant --")
    print("     try regenerating features without it for the final.")
    print("  4. The RF-CV best (numTrees, maxDepth) tells you whether the")
    print("     grid was at a corner: if it picked the max value of a param,")
    print("     extend the grid in that direction next run.")
    print("  5. hw9_probabilities.csv is now ready as input to the Final.")

    print(f"\n  Total PySpark time: {elapsed(t_spark)}")
    spark.stop()
```

```python


def rank_average_blend(model_names, results_dict):
    """Within each user, convert each model's probs to ranks (0..5), then
    average ranks across models.  Returns {(uid,tid): avg_rank}."""
    by_user_per_model = {n: defaultdict(list) for n in model_names}
    for n in model_names:
        for (uid, tid), p in results_dict[n][1].items():
            by_user_per_model[n][uid].append((tid, p))

    avg_rank = {}
    sample_user = next(iter(by_user_per_model[model_names[0]]))
    all_users   = set(by_user_per_model[model_names[0]].keys())
    for uid in all_users:
        rank_sum = defaultdict(float)
        for n in model_names:
            ranked = sorted(by_user_per_model[n][uid], key=lambda x: x[1])
            for r, (tid, _) in enumerate(ranked):
                rank_sum[tid] += r
        for tid, rs in rank_sum.items():
            avg_rank[(uid, tid)] = rs / len(model_names)
    return avg_rank


# ---------------------------------------------------------------------------
# 7. Main
# ---------------------------------------------------------------------------

def main():
    tee = Tee(RESULTS_FILE)
    sys.stdout = tee
    t_total = time.time()

    try:
        section("EE627 HOMEWORK 9 - PySpark Music Recommender")
        print(f"  Log file: {RESULTS_FILE}")
        print(f"  Started : {time.strftime('%Y-%m-%d %H:%M:%S')}")

        section("LOADING RAW DATA")
        user_ratings = parse_training(TRAIN_FILE)
        track_meta   = parse_tracks(TRACK_FILE)
        album_meta   = parse_albums(ALBUM_FILE)
        test_users   = parse_test(TEST_FILE)
        labels       = parse_labels(LABEL_FILE)

        section("ENRICHMENT & AUXILIARY STRUCTURES")
        enriched = enrich_tracks(track_meta, album_meta)
        global_stats = build_global_item_stats(user_ratings)
        album_to_tracks, artist_to_tracks = build_track_mappings(enriched)

        section("BUILDING FEATURE CSVs")
        if not os.path.exists(FEATURE_CSV):
            build_labeled_csv(user_ratings, enriched, global_stats,
                              album_to_tracks, artist_to_tracks,
                              labels, FEATURE_CSV)
        else:
            print(f"  Labeled CSV already exists, skipping rebuild.")
            print(f"  (Delete {FEATURE_CSV} to force rebuild)")

```

```
        if not os.path.exists(TEST_CSV):
            build_test_csv(user_ratings, enriched, global_stats,
                           album_to_tracks, artist_to_tracks,
                           test_users, TEST_CSV)
        else:
            print(f"  Test CSV already exists, skipping rebuild.")
            print(f"  (Delete {TEST_CSV} to force rebuild)")

        run_pyspark(FEATURE_CSV, TEST_CSV, SCRIPT_DIR)

        section("COMPLETE")
        print(f"  Total wall time : {elapsed(t_total)}")
        print(f"  Full log saved  : {RESULTS_FILE}")

    finally:
        tee.close()


if __name__ == "__main__":
    main()
```

Listing 1: EE627_HW9_PySpark.py — Five PySpark.ML classifiers, six ensemble blends, and a 9-column probability matrix saved for the final project.

9 Appendix: Console Output

```

==================================================================
  EE627 HOMEWORK 9 - PySpark Music Recommender
==================================================================
  Log file: d:\judee\Google Drive\School\Classes\EE627\Homework\Homework 9\results_hw9.txt
  Started : 2026-05-03 20:51:37

==================================================================
  LOADING RAW DATA
==================================================================
Loading training data ...
  49,204 users  |  12,403,575 ratings  [3.2s]
  Ratings per user : min=17  median=61  max=107936
  Rating values    : min=0  mean=49.77  max=100
Loading track metadata ...
  224,041 tracks  [0.3s]
  Missing album : 18,509  |  missing artist: 26,896
  Genres per track: min=0  mean=2.4  max=21
Loading album metadata ...
  52,829 albums  [0.1s]
Loading test candidates ...
  20,000 test users  |  120,000 total candidates  [0.0s]
  Candidates per user: min=6  max=6  (expected 6 each)
Loading ground-truth labels ...
  6,000 rows  |  1,000 users  [0.0s]
  Label balance: 3,000 positives (50.0%)  |  3,000 negatives (50.0%)

==================================================================
  ENRICHMENT & AUXILIARY STRUCTURES
==================================================================
Enriching tracks with album metadata ...
  Artists filled via album : 8,134
  Genres added via album   : 119,842
  Tracks still missing artist after enrichment: 18,762  [0.3s]
  Global item stats: 295,799 items  [2.2s]
  Raters per item  : min=1  median=10  max=23364
  Global avg rating: min=0.0  mean=44.06  max=100.0
  Album->tracks  : 31,141 albums  (avg 6.6 tracks/album)
  Artist->tracks : 12,291 artists  (avg 16.7 tracks/artist)

==================================================================
  BUILDING FEATURE CSVs
==================================================================
  Labeled CSV already exists, skipping rebuild.
  (Delete d:\judee\Google Drive\School\Classes\EE627\Homework\Homework 9\features_labeled.csv to force rebuild )
  Test CSV already exists, skipping rebuild.
  (Delete d:\judee\Google Drive\School\Classes\EE627\Homework\Homework 9\features_test.csv to force rebuild)

==================================================================
  LOADING & PREPARING DATA IN SPARK
==================================================================
  Labeled rows : 6,000  (3,000 pos = 50.0%  |  3,000 neg = 50.0%)
  Full labeled : 6,000  (used to refit final models)
  Train (70%)  : 4,260 rows  (used for CV & tuning)
  Validation   : 1,740 rows  (858 pos = 49.3%)
```

```
  Kaggle test  : 120,000 rows

================================================================
  [1/7] LOGISTIC REGRESSION (elastic-net CV)
================================================================
  Grid size: 9 combos x 3 folds = 27 fits

  [LR] Validation metrics:
    AUC-ROC   : 0.9376
    Accuracy  : 0.8707
    Precision : 0.8718
    Recall    : 0.8707
    F1        : 0.8705
    Confusion matrix (val set):
      Predicted ->     0       1
      Actual 0       794      88   (spec=0.900)
      Actual 1       137     721   (sens=0.840)

  [LR-CV] Best hyperparameters:
    regParam        = 0.01
    elasticNetParam = 0.5

  [LR] Top-10 coefficients (|beta|):
  Rank  Feature                    Coefficient
  ----- ------------------------- ------------
  1     has_album                +     1.2034
  2     has_artist               +     1.0638
  3     cf_count_log             +     0.3895
  4     user_n_log                    -0.3822
  5     art_nrat_log                  -0.1855
  6     artist_score             +     0.0107
  7     genre_above_mean         +     0.0107
  8     album_score              +     0.0090
  9     cf_score                 +     0.0087
  10    genre_nrat_log                -0.0082

  [LR] Refitting on full 6,000 rows with best params...
  LR-CV training time: 19.3s

================================================================
  [2/7] DECISION TREE CLASSIFIER
================================================================

  [DT] Validation metrics:
    AUC-ROC   : 0.7899
    Accuracy  : 0.8707
    Precision : 0.8725
    Recall    : 0.8707
    F1        : 0.8705
    Confusion matrix (val set):
      Predicted ->     0       1
      Actual 0       801      81   (spec=0.908)
      Actual 1       144     714   (sens=0.832)

  [DT] Feature importances (all 25 features):
  Rank  Feature                     Importance  Bar
  ----- ------------------------- ------------  --------------------
  1     v2_score                      0.791961  ####################
  2     cf_score                      0.060152  #
```

```
  3     alb_global_avg               0.056703  #
  4     has_album                    0.030360
  5     artist_score                 0.012654
  6     user_n_log                   0.012541
  7     art_nrat_log                 0.012454
  8     genre_above_mean             0.008980
  9     genre_mean                   0.005721
  10    user_mean                    0.003798    <- low signal
  11    alb_nrat_log                 0.002805    <- low signal
  12    user_std                     0.001105    <- low signal
  13    album_score                  0.000767    <- low signal
  14    has_artist                   0.000000    <- low signal
  15    genre_count                  0.000000    <- low signal
  16    genre_max                    0.000000    <- low signal
  17    genre_min                    0.000000    <- low signal
  18    genre_var                    0.000000    <- low signal
  19    album_artist_int             0.000000    <- low signal
  20    album_above_mean             0.000000    <- low signal
  21    artist_above_mean            0.000000    <- low signal
  22    art_global_avg               0.000000    <- low signal
  23    genre_global_avg             0.000000    <- low signal
  24    genre_nrat_log               0.000000    <- low signal
  25    cf_count_log                 0.000000    <- low signal

  DT depth used : 5  |  nodes: 41
  DT training time: 1.6s

================================================================
  [3/7] RANDOM FOREST CLASSIFIER (Big CV-tuned)
================================================================
  Grid size: 12 combos x 5 folds = 60 fits

  [RF] Validation metrics:
    AUC-ROC   : 0.9468
    Accuracy  : 0.8799
    Precision : 0.8800
    Recall    : 0.8799
    F1        : 0.8799
    Confusion matrix (val set):
      Predicted ->     0       1
      Actual 0       786      96   (spec=0.891)
      Actual 1       113     745   (sens=0.868)

  [RF-CV] Best hyperparameters:
    numTrees             = 200
    maxDepth             = 10
    featureSubsetStrategy= sqrt

  [GBT-CV] Per-parameter-combination average AUC (12 combinations):
    Combo  1: 0.9411
    Combo  2: 0.9411
    Combo  3: 0.9401
    Combo  4: 0.9401
    Combo  5: 0.9386
    Combo  6: 0.9386
    Combo  7: 0.9400
    Combo  8: 0.9400
    Combo  9: 0.9395
    Combo 10: 0.9395
```

```
    Combo 11: 0.9381
    Combo 12: 0.9381
  Best combination index: 1  (AUC = 0.9411)

  [RF-CV best] Feature importances (all 25 features):
  Rank  Feature                     Importance  Bar
  ----- ------------------------- ------------  --------------------
  1     v2_score                      0.185702  ####################
  2     artist_score                  0.151731  ################
  3     artist_above_mean             0.092136  #########
  4     has_artist                    0.076347  ########
  5     has_album                     0.056338  ######
  6     cf_score                      0.050034  #####
  7     album_artist_int              0.046900  #####
  8     album_score                   0.038368  ####
  9     cf_count_log                  0.036002  ###
  10    album_above_mean              0.029379  ###
  11    user_n_log                    0.027984  ###
  12    art_nrat_log                  0.027707  ##
  13    alb_nrat_log                  0.023582  ##
  14    alb_global_avg                0.023078  ##
  15    art_global_avg                0.020566  ##
  16    user_std                      0.018233  #
  17    genre_global_avg              0.017614  #
  18    genre_nrat_log                0.015341  #
  19    user_mean                     0.015305  #
  20    genre_above_mean              0.014625  #
  21    genre_min                     0.009100
  22    genre_mean                    0.007673
  23    genre_max                     0.007015
  24    genre_count                   0.005276
  25    genre_var                     0.003964    <- low signal

  [RF] Refitting on full 6,000 rows with best params...
  RF-CV+refit training time: 9.0m

================================================================
  [4/7] GRADIENT-BOOSTED TREE (base)
================================================================

  [GBT] Validation metrics:
    AUC-ROC   : 0.9377
    Accuracy  : 0.8747
    Precision : 0.8754
    Recall    : 0.8747
    F1        : 0.8746
    Confusion matrix (val set):
      Predicted ->     0       1
      Actual 0       793      89   (spec=0.899)
      Actual 1       129     729   (sens=0.850)

  [GBT] Feature importances (all 25 features):
  Rank  Feature                     Importance  Bar
  ----- ------------------------- ------------  --------------------
  1     v2_score                      0.479098  ####################
  2     user_n_log                    0.068544  ##
  3     has_album                     0.045867  #
  4     cf_count_log                  0.044018  #
  5     art_nrat_log                  0.036587  #
```

```
  6     genre_above_mean              0.035736  #
  7     user_mean                     0.032099  #
  8     user_std                      0.031707  #
  9     alb_global_avg                0.030680  #
  10    has_artist                    0.026640  #
  11    art_global_avg                0.025452  #
  12    cf_score                      0.025382  #
  13    genre_global_avg              0.020018
  14    genre_nrat_log                0.019047
  15    alb_nrat_log                  0.013404
  16    genre_mean                    0.011914
  17    genre_var                     0.011432
  18    artist_above_mean             0.011192
  19    artist_score                  0.008228
  20    genre_min                     0.007413
  21    genre_count                   0.007403
  22    album_score                   0.003497    <- low signal
  23    album_artist_int              0.001715    <- low signal
  24    album_above_mean              0.001680    <- low signal
  25    genre_max                     0.001246    <- low signal
  GBT training time: 2.8s

================================================================
  [5/7] GBT + BIG CROSS-VALIDATION TUNING
================================================================
  Grid size: 24 combos x 5 folds = 120 fits

  [GBT-CV] Validation metrics:
    AUC-ROC   : 0.9454
    Accuracy  : 0.8828
    Precision : 0.8828
    Recall    : 0.8828
    F1        : 0.8827
    Confusion matrix (val set):
      Predicted ->     0       1
      Actual 0       787      95   (spec=0.892)
      Actual 1       109     749   (sens=0.873)

  [GBT-CV] Best hyperparameters:
    maxDepth        = 3
    maxIter         = 200
    stepSize        = 0.05
    subsamplingRate = 0.7

  [GBT-CV] Per-parameter-combination average AUC (24 combinations):
    Combo  1: 0.9382
    Combo  2: 0.9373
    Combo  3: 0.9391
    Combo  4: 0.9390
    Combo  5: 0.9398
    Combo  6: 0.9391
    Combo  7: 0.9395
    Combo  8: 0.9404
    Combo  9: 0.9405
    Combo 10: 0.9403
    Combo 11: 0.9386
    Combo 12: 0.9395
    Combo 13: 0.9369
    Combo 14: 0.9361
```

```
    Combo 15: 0.9347
    Combo 16: 0.9369
    Combo 17: 0.9370
    Combo 18: 0.9358
    Combo 19: 0.9336
    Combo 20: 0.9356
    Combo 21: 0.9351
    Combo 22: 0.9341
    Combo 23: 0.9313
    Combo 24: 0.9328
  Best combination index: 9  (AUC = 0.9405)

  [GBT-CV best] Feature importances (all 25 features):
  Rank  Feature                      Importance  Bar
  ----- ------------------------- ------------  --------------------
  1     v2_score                       0.193430  ####################
  2     user_n_log                     0.119379  ############
  3     has_album                      0.086732  ########
  4     art_nrat_log                   0.069368  #######
  5     genre_above_mean               0.052425  #####
  6     cf_count_log                   0.046808  ####
  7     genre_global_avg               0.041737  ####
  8     art_global_avg                 0.041353  ####
  9     alb_nrat_log                   0.037824  ###
  10    alb_global_avg                 0.035564  ###
  11    genre_nrat_log                 0.031668  ###
  12    user_mean                      0.030985  ###
  13    user_std                       0.028620  ##
  14    album_score                    0.028467  ##
  15    artist_above_mean              0.024919  ##
  16    artist_score                   0.024022  ##
  17    cf_score                       0.020175  ##
  18    has_artist                     0.018920  #
  19    genre_count                    0.013530  #
  20    album_above_mean               0.012456  #
  21    genre_min                      0.011324  #
  22    genre_var                      0.009637
  23    album_artist_int               0.009451
  24    genre_mean                     0.006765
  25    genre_max                      0.004441    <- low signal

  [GBT-CV] Refitting on full 6,000 rows with best params...
  GBT-CV+refit training time: 21.1m

================================================================
  [6/7] RF WITHOUT v2_score (raw-feature variant)
================================================================
  Features used: 24 (dropped v2_score)

  [RF-no-v2] Validation metrics:
    AUC-ROC   : 0.9472
    Accuracy  : 0.8874
    Precision : 0.8874
    Recall    : 0.8874
    F1        : 0.8873
    Confusion matrix (val set):
      Predicted ->     0       1
      Actual 0       787      95   (spec=0.892)
      Actual 1       101     757   (sens=0.882)
```

```

  [RF-no-v2] Feature importances (all 24 features):
  Rank  Feature                     Importance  Bar
  ----- ------------------------- ------------  --------------------
  1     artist_score                  0.202663  ####################
  2     artist_above_mean             0.129859  ############
  3     has_artist                    0.108696  ##########
  4     has_album                     0.066939  ######
  5     album_artist_int              0.057955  #####
  6     album_score                   0.056707  #####
  7     cf_score                      0.051324  #####
  8     cf_count_log                  0.043127  ####
  9     album_above_mean              0.039105  ###
  10    user_n_log                    0.030229  ##
  11    art_nrat_log                  0.028340  ##
  12    alb_nrat_log                  0.024064  ##
  13    art_global_avg                0.023561  ##
  14    user_std                      0.018128  #
  15    alb_global_avg                0.017612  #
  16    user_mean                     0.016359  #
  17    genre_global_avg              0.016177  #
  18    genre_nrat_log                0.015266  #
  19    genre_above_mean              0.013326  #
  20    genre_min                     0.011152  #
  21    genre_mean                    0.010193  #
  22    genre_max                     0.010073
  23    genre_count                   0.005764
  24    genre_var                     0.003380    <- low signal
  RF-no-v2 training time: 8.4s

================================================================
  [7/7] EXTRA TREES (RF, more randomness)
================================================================

  [ExtraT] Validation metrics:
    AUC-ROC   : 0.9450
    Accuracy  : 0.8764
    Precision : 0.8765
    Recall    : 0.8764
    F1        : 0.8764
    Confusion matrix (val set):
      Predicted ->     0       1
      Actual 0       780     102   (spec=0.884)
      Actual 1       113     745   (sens=0.868)

  [ExtraT] Feature importances (all 25 features):
  Rank  Feature                     Importance  Bar
  ----- ------------------------- ------------  --------------------
  1     v2_score                      0.217076  ####################
  2     artist_score                  0.123139  ###########
  3     has_artist                    0.089826  ########
  4     artist_above_mean             0.076723  #######
  5     user_n_log                    0.045489  ####
  6     cf_score                      0.036741  ###
  7     art_nrat_log                  0.035619  ###
  8     user_std                      0.035016  ###
  9     alb_nrat_log                  0.034923  ###
  10    alb_global_avg                0.032829  ###
  11    genre_global_avg              0.032064  ##
```

```
  12    user_mean                    0.031802  ##
  13    genre_nrat_log               0.030905  ##
  14    art_global_avg               0.030572  ##
  15    has_album                    0.028665  ##
  16    cf_count_log                 0.025519  ##
  17    album_score                  0.022395  ##
  18    genre_above_mean             0.019096  #
  19    album_above_mean             0.013543  #
  20    album_artist_int             0.010252
  21    genre_count                  0.007036
  22    genre_min                    0.006202
  23    genre_max                    0.006010
  24    genre_mean                   0.005132
  25    genre_var                    0.003425    <- low signal
  ExtraT training time: 55.5s

================================================================
  VALIDATION AUC SUMMARY
================================================================
  Classifier                   Val AUC   vs best
  -------------------------- ---------  --------
  RF without v2_score           0.9472   +0.0000  <- BEST
  Random Forest (Big CV)        0.9468   -0.0004
  GBT (Big CV)                  0.9454   -0.0018
  Extra Trees                   0.9450   -0.0022
  GBT (base)                    0.9377   -0.0095
  Logistic Regression CV        0.9376   -0.0096
  Decision Tree                 0.7899   -0.1573

================================================================
  WRITING PER-CLASSIFIER KAGGLE SUBMISSIONS
================================================================
  lr                     (AUC=0.9376)  60,000 rec / 60,000 not rec  -> submission_hw9_lr.csv
  dt                     (AUC=0.7899)  60,000 rec / 60,000 not rec  -> submission_hw9_dt.csv
  rf                     (AUC=0.9468)  60,000 rec / 60,000 not rec  -> submission_hw9_rf.csv
  gbt                    (AUC=0.9377)  60,000 rec / 60,000 not rec  -> submission_hw9_gbt.csv
  gbt_cv                 (AUC=0.9454)  60,000 rec / 60,000 not rec  -> submission_hw9_gbt_cv.csv
  rf_no_v2               (AUC=0.9472)  60,000 rec / 60,000 not rec  -> submission_hw9_rf_no_v2.csv
  extra                  (AUC=0.9450)  60,000 rec / 60,000 not rec  -> submission_hw9_extra.csv

================================================================
  ENSEMBLE BLENDS
================================================================

  Building rf_gbt_blend (RF 0.5 + GBT-CV 0.5)...
  blend_rf_gbt           (RF + GBT-CV equal)  60,000 rec / 60,000 not rec  -> submission_hw9_blend_rf_gbt.csv
  Building rf_heavy_blend (RF 2 + GBT-CV 1)...
  blend_rf_heavy         (RF 2 + GBT-CV 1)  60,000 rec / 60,000 not rec  -> submission_hw9_blend_rf_heavy.csv
  Building three_model_blend (RF + GBT-CV + LR equal)...
  blend_three            (RF + GBT-CV + LR)  60,000 rec / 60,000 not rec  -> submission_hw9_blend_three.csv
  Building tree_ensemble (RF + GBT-CV + Extra + RF-no-v2)...
  blend_trees            (4-tree ensemble)  60,000 rec / 60,000 not rec  -> submission_hw9_blend_trees.csv
  Building mega_blend (all 7 classifiers, AUC-weighted)...
  blend_mega             (ALL, AUC-weighted)  60,000 rec / 60,000 not rec  -> submission_hw9_blend_mega.csv
  Building rank_blend (rank-averaged across models)...
  blend_rank             (rank-avg of 4 trees)  60,000 rec / 60,000 not rec  -> submission_hw9_blend_rank.csv

================================================================
  SAVING PROBABILITY MATRIX FOR FINAL PROJECT
```

```
================================================================
  Wrote 120,000 rows x 7 models -> hw9_probabilities.csv
  This file is the input matrix for the Final project meta-learner.

================================================================
  IMPROVEMENT OPPORTUNITIES
================================================================
  After this run, the natural next steps are:
  1. Submit blend_rank, blend_trees, blend_rf_heavy first --
     these are the highest-EV blends (diverse signal sources).
  2. Compare blend Kaggle scores to the best single model --
     if mega_blend > rf alone, ensembling is paying off.
  3. If RF-no-v2 scored well, it means v2_score was redundant --
     try regenerating features without it for the final.
  4. The RF-CV best (numTrees, maxDepth) tells you whether the
     grid was at a corner: if it picked the max value of a param,
     extend the grid in that direction next run.
  5. hw9_probabilities.csv is now ready as input to the Final.

  Total PySpark time: 32.1m

================================================================
  COMPLETE
================================================================
  Total wall time : 32.2m
  Full log saved  : d:\judee\Google Drive\School\Classes\EE627\Homework\Homework 9\results_hw9.txt
```

Listing 2: Run log from the production HW9 run.